

Marketing Hub 升级计划书

版本：V4 团队执行版

适用对象：产品、设计、前端、后端、AI 工程、运维、测试、市场增长团队

目标周期：16 周内完成从“初版可演示产品”到“可小规模商业验证 SaaS”的升级

0. 执行摘要

Marketing Hub 当前已经具备一个 AI 营销工作台的雏形：React + Vite 前端、Django + DRF 后端、项目/活动/ workflow 草稿/社区作品/生成任务/用量记录/BYOK 配置/订阅方案等基础模块均已存在。这个基础值得继续投入，因为它已经不是单纯的静态 Demo，而是有了“项目制内容生产”的数据骨架。

但当前版本还不能直接经受公开市场考验。主要原因不是功能数量少，而是“生产可信度”不足：登录是 demo token，AI 密钥未按组织隔离且缺少加密托管，workflow 画布仍是手写交互，任务运行、素材存储、RAG 检索、计费、权限、监控、安全与测试都还处在初级阶段。若直接面向真实付费用户，最容易暴露的问题会是：生成结果不可追踪、资产不可沉淀、多人协作不可控、成本不可预测、异常不可解释、数据安全不可证明。

升级方向必须从“AI 工具集合”升级为“面向营销团队的内容生产操作系统”。核心不是再堆更多生成入口，而是把品牌记忆、workflow 编排、资产管理、多人审阅、渠道发布、用量成本和商业计费连接成闭环。

本计划建议的产品定位：

- 面向中小品牌、电商运营团队、内容营销团队、MCN/代理商的 AI 营销内容生产工作台。
- 用项目和活动管理营销上下文，用 workflow 复用生产 SOP，用品牌记忆保证一致性，用资产库沉淀可复用素材，用团队协作和审阅控制质量，用成本账本支撑商业化。
- 第一阶段不追求成为全渠道发布平台，而是先打穿“从 brief 到多平台内容包”的核心生产闭环。

1. 当前项目现状诊断

1.1 已经具备的真实基础

当前代码库中已经存在以下可继续演进的基础能力：

- 前端技术栈：React 19、Vite、TypeScript、Tailwind CSS、lucide-react。
- 后端技术栈：Django 6、Django REST Framework、Celery、Redis、PostgreSQL/SQLite fallback。
- 领域拆分：后端已经拆出 `accounts`、`workspaces`、`generation`、`community`、`ai_gateway`、`billing` 等 app，`api` 作为模型与兼容层。
- 项目制数据模型：`Organization`、`Membership`、`Project`、`Campaign` 已存在。
- workflow 数据模型：`WorkspaceDraft`、`WorkflowTemplate` 已存在，支持 `nodes`、`edges`、`viewport`、`selected node`、`last run summary`。
- 任务账本：`GenerationTask` 支持 `queued/running/succeeded/failed`，记录 `payload`、`result`、`token`、`cost`、`celery_task_id`。
- 资产沉淀：`Asset` 模型存在，可关联 `organization/project/campaign`。
- 商业化雏形：`PLAN_LIMITS`、`BillingPlansView`、`AIConfiguration`、`BYOK` 字段已存在。
- 社区/灵感雏形：`CommunityCreation`、点赞、关键词检索 fallback 已存在。
- workflow 画布雏形：前端 `WorkflowBuilder` 已支持节点拖动、缩放、连接、右键菜单、自定义智能体、`schema warning`、模板加载、运行、重试。
- 项目管理雏形：前端 `ProjectManager` 已支持项目创建、筛选、文件夹字段、平台标签、状态标签、品牌上下文编辑。
- Docker Compose：已具备 PostgreSQL、Redis、后端、前端、worker 的基础编排。

1.2 当前版本的关键短板

这些短板会直接影响市场化、团队协作和生产环境稳定性：

- 身份认证仍是 demo 级别：登录接口返回 `demo-session-token-*`，没有真正 token 校验、session 策略、刷新机制、退出机制、密码策略、组织级权限拦截。

- 权限边界不完整: `Membership` 已有, 但 API 层没有系统性使用 RBAC 检查, 跨组织访问、资源归属校验、只读/编辑/管理员边界不足。
- AI 配置未组织隔离: `AIConfiguration` 当前是全局配置, 真实 SaaS 必须按组织、环境和 provider 隔离。
- API Key 安全不足: 密钥以模型字段保存, 缺少字段级加密、KMS/Vault、密钥轮换、访问审计和最小权限策略。
- CORS/Debug 默认偏开发环境: `CORS_ALLOW_ALL_ORIGINS` 和 `DEBUG` 的默认值不适合生产。
- 计费只是方案展示和项目数量限制: 没有支付订阅、额度扣减、并发限制、模型成本实时估算、超额策略、发票和退款流程。
- RAG 不是语义检索: 当前是关键词 fallback, 不能支撑“品牌记忆”“素材对齐”“历史风格复用”的市场承诺。
- 媒体资产未真正生产级持久化: 图片和音频输出仍依赖预览/外链/fallback 路径, 缺少对象存储、CDN、预签名上传、转码、缩略图和生命周期管理。
- 前端入口过于集中: `App.tsx` 承载大量业务状态, 后续会影响可维护性、测试和多人协作。
- 工作流引擎仍是浅层 DAG: 节点 I/O schema 只是基础结构, 缺少强类型校验、条件分支、错误恢复、运行快照、版本化、长任务可观测性。
- 缺少前端数据缓存层: 没有 TanStack Query/SWR 级别的请求缓存、失效策略、乐观更新和重试策略。
- 缺少完整质量体系: 后端测试覆盖核心路径较少, 前端缺少单元测试、组件测试、端到端测试、可访问性测试和视觉回归。
- 缺少生产可观测性: 没有 Sentry、OpenTelemetry、结构化日志、Prometheus/Grafana、任务队列监控和告警。
- 市场闭环不足: 用户能够生成内容, 但还没有形成“品牌输入 -> 多平台内容包 -> 审阅 -> 发布/导出 -> 效果反馈 -> 模板复用”的闭环。

2. 市场可行性判断

2.1 现在能否经得起市场考验

结论: 当前版本适合内部演示、种子用户访谈和封闭测试, 不适合直接开放付费市场。

可用于市场验证的部分：

- 可以展示 AI 营销工作台的完整方向。
- 可以验证用户是否需要“项目制 + 品牌记忆 + workflows”的组合。
- 可以用于收集不同人群对文案、图片、分镜、配音四类入口的偏好。
- 可以验证团队是否愿意把营销 brief 和历史素材沉淀到系统中。

不能直接商业化的部分：

- 不能承诺企业级数据安全。
- 不能承诺多用户协作权限隔离。
- 不能承诺稳定的真实 AI 媒体生成。
- 不能承诺成本可控的订阅消耗。
- 不能承诺 RAG/品牌记忆达到生产级准确度。
- 不能承诺高并发和任务稳定性。

2.2 目标用户分层

第一优先级用户：

- 电商和消费品牌运营团队：需要高频产出小红书、抖音、公众号、视频号等内容。
- 中小营销代理商：需要把客户 brief 快速转成多平台内容包，并复用成功模板。
- 内容团队负责人：需要统一品牌调性、降低重复沟通、沉淀内容资产。

第二优先级用户：

- 独立创作者和小团队：需要低成本生成脚本、图文和配音，但付费能力较弱。
- 企业市场部：需求强，但采购周期、权限、安全、审计要求更高。

暂不优先：

- 只想生成单条文案的轻量用户。这个人群会被通用大模型和低价工具覆盖，留存与付费难度高。
- 复杂广告投放优化用户。当前系统还没有投放数据、归因和预算优化能力。

2.3 市场竞争中的差异化

Marketing Hub 不应定位为“另一个 AI 文案工具”，否则会被通用聊天产品、设计工具和内容平台插件挤压。可防守的差异化应来自以下组合：

- 项目制上下文：每个品牌、活动和渠道都沉淀独立上下文。
- 可复用工作流：把团队的内容 SOP 变成模板，而不是每次从空白 prompt 开始。
- 品牌记忆：历史内容、品牌规范、禁用词、视觉风格、爆款案例能够持续影响生成。
- 多角色协作：运营、设计、审核、客户、管理员在同一工作台完成流转。
- 成本和合规可控：组织可以选择平台额度或 BYOK，并看到每次任务的成本和输出。
- 从生成到资产：产物不是一次性复制走，而是自动进入项目资产库、可检索、可复用、可审计。

2.4 商业化判断

可行的早期收费模式：

- Free：个人试用，项目数和月生成额度受限，不开放高级工作流和团队权限。
- Pro：面向小团队，开放更多项目、工作流模板、品牌记忆、基础素材库和 BYOK 抵扣。
- Team：按 seat 或组织收费，开放协作、审阅、模板库、批量生成、团队资产库。
- Enterprise：SSO、私有部署/VPC、审计日志、自定义模型网关、数据保留策略、专属 SLA。

不建议早期采用的收费模式：

- 单纯按生成次数收费。用户会直接和模型原始 API 价格比较，价值感弱。
- 过早做模板市场分成。没有足够活跃供给和需求前，市场会显得空。
- 过早承诺全渠道自动发布。各平台 API、风控、账号授权和内容审核会拉长周期。

2.5 市场验证指标

封闭测试阶段必须跟踪以下指标：

- 激活率：新组织在 24 小时内完成首个项目、首个品牌上下文、首个生成任务的比例。
- 首次价值时间：从登录到拿到可用内容包的中位时间，目标小于 8 分钟。
- workflow 复用率：用户是否保存并复用模板，目标测试期内超过 30% 的活跃组织复用至少 1 次。
- 项目留存：每个组织是否创建第二个项目或第二个活动。
- 资产沉淀率：生成内容进入资产库并被再次打开/复用的比例。
- 协作邀请率：团队版价值验证，目标超过 20% 的组织邀请第二名成员。
- 生成成功率：任务成功率目标大于 95%，失败任务必须可解释、可重试。
- 付费意愿：愿意为团队协作、品牌记忆、批量生成、BYOK 成本控制付费的访谈比例。

3. 产品升级总目标

3.1 一句话目标

把 Marketing Hub 升级为“营销团队从 brief 到多平台内容包的 AI 生产工作台”。

3.2 三个核心闭环

内容生产闭环：

1. 创建品牌/项目/活动。
2. 填写品牌记忆、目标受众、卖点、渠道和禁用要求。
3. 选择模板或搭建工作流。
4. 批量生成文案、图片提示词、分镜、口播、配音。
5. 审阅、修改、重试、版本对比。
6. 导出或进入待发布队列。

资产沉淀闭环：

1. 每次生成自动形成资产。
2. 资产绑定项目、活动、渠道、标签、使用状态。
3. 历史资产进入品牌记忆和灵感检索。
4. 成功资产可沉淀为模板或案例。

商业计费闭环：

1. 组织选择套餐。
2. 平台模式消耗额度，BYOK 模式记录调用但减少平台成本。
3. 每次任务记录 token、模型、耗时、成本、失败原因。
4. 管理员看到额度、账单、成员使用排行和异常消耗。

3.3 产品原则

- 运营人员优先：默认界面讲业务语言，不把模型参数、RAG、token、任务 ID 暴露给普通用户。
- 可解释但不打扰：普通用户看到进度、结果和建议；高级用户可打开调试面板看日志。
- 少即是多：早期聚焦“品牌记忆 + 多平台内容包 + workflow 复用”，不要分散到 CRM、投放归因、社交私信等领域。
- 数据必须可沉淀：所有生成结果、上下文、修改意见、失败原因都要被结构化保存。
- 成本必须可控：每个组织、项目、成员、任务都能追踪用量和预算。
- 安全从第一天补齐：密钥、组织数据、资产访问和成员权限不能留到后期。

4. 人机交互体验提升计划

4.1 信息架构重构

建议从当前 tab 式功能集合升级为稳定 App Shell：

- 顶部：组织切换、全局搜索、任务队列、通知、账号菜单。

- 左侧导航：数据看板、项目、 workflow、资产库、灵感库、社区/模板、计费、设置。
- 主区域：根据当前项目展示上下文，不让用户在多个孤立功能页之间迷失。
- 右侧上下文面板：选中项目、节点、资产或任务时展示配置、历史、评论、用量。

项目内二级结构：

- 总览：项目 brief、活动目标、最近任务、关键资产、待处理事项。
- 品牌记忆：品牌调性、受众、卖点、禁用词、参考案例、视觉规范。
- 内容生产：快速生成、批量生成、workflow 运行。
- workflow：画布、模板、运行记录。
- 资产：图片、音频、文案、分镜、文档。
- 审阅：待审核内容、评论、版本对比。
- 设置：成员、标签、渠道、权限。

4.2 文案和术语去技术化

面向普通运营人员：

- “RAG 搜索”改为“品牌灵感搜索”或“历史素材检索”。
- “Generation Task”改为“生成任务”。
- “I/O Schema”默认隐藏，改为“输入要求”和“输出内容”。
- “Token/Cost”放入管理员或用量详情，不在生成主流程打断用户。
- “Celery Task ID”只在调试面板出现。
- “BYOK”在界面解释为“使用自己的模型密钥”。

加载状态建议：

- 排队中：正在排队处理，本次任务预计需要约 xx 秒。
- 生成中：正在根据品牌记忆生成内容。
- 检索中：正在查找相似历史素材。
- 失败：说明失败原因，并提供重试、换模型、减少输入长度、联系管理员。

4.3 首次使用体验

新增 5 步 onboarding:

1. 选择使用场景：新品上市、内容日更、短视频脚本、品牌活动、代理商客户项目。
2. 创建品牌：品牌名称、行业、目标人群、语调、禁用词、参考链接。
3. 选择渠道：小红书、抖音、公众号、视频号、B 站等。
4. 选择起始模板：图文种草、短视频脚本、活动预热、直播预告、产品卖点拆解。
5. 生成第一份内容包：一次输出标题、正文、标签、图片提示词、分镜或口播。

验收标准：

- 新用户不阅读文档也能完成第一轮生成。
- 空状态必须有明确下一步动作。
- 首次创建项目不要求填写过多字段，允许后续补全。

4.4 项目管理体验

当前 `ProjectManager` 已有文件夹、平台标签、状态标签的基础，应继续升级：

- 文件夹从字符串字段升级为独立 `Folder` 模型，支持层级、排序、权限、归档。
- 标签体系拆分为平台标签、内容类型标签、状态标签、自定义标签。
- 支持批量移动、批量归档、批量打标签、批量导出。
- 支持列表/看板/网格三种视图。
- 项目搜索支持名称、brief、品牌上下文、资产标题、标签。
- 项目卡片展示最近活动、待审内容数、最近生成状态、消耗额度。
- 删除动作必须二次确认，默认建议归档而非删除。

4.5 工作流画布体验

当前手写画布可以用于验证概念，但生产级建议迁移到 XYFlow/React Flow。

必须升级的交互：

- 节点拖拽、缩放、吸附、框选、多选、复制粘贴、撤销重做。
- 输入/输出端口明确展示，连接时只允许兼容类型连接。
- 节点配置使用右侧属性面板，不依赖大弹窗。
- 运行时展示节点状态：未运行、排队、运行、成功、失败、跳过。
- 支持从失败节点重试，支持带修改意见重试。
- 支持运行预览：运行前展示预计调用节点、预计成本、预计耗时。
- 支持版本历史：每次保存和运行形成可回滚版本。
- 支持模板化：把一条成熟 workflow 保存成组织模板或公共模板。
- 支持只读分享：给客户或审核者查看结果而不允许修改结构。

节点类型升级：

- 品牌上下文节点：输出品牌摘要、语调、受众、禁用词。
- 文案节点：输出标题、正文、标签、CTA、多平台变体。
- 图片提示词节点：输出结构化 prompt、负面 prompt、画幅、风格。
- 图片生成节点：连接真实图像模型，输出图片资产。
- 分镜节点：输出镜头、时长、画面、口播、转场。
- 配音节点：输出音频资产和字幕时间轴。
- 检索节点：从品牌记忆/资产库中检索参考。
- 审核节点：检查敏感词、品牌一致性、渠道规则。
- 自定义智能体：允许设置 prompt、输入字段、输出 schema、模型、温度、失败策略。

4.6 生成体验

生成主流程必须从“单点功能”升级为“内容包”：

- 用户输入一个 brief，系统按渠道生成完整内容包。
- 每个内容包包含：平台、标题、正文、标签、图片建议、分镜/口播建议、审核建议、导出格式。
- 支持对结果做“更短、更有冲突感、更专业、更年轻化、减少夸张表达”等快捷改写。

- 支持保留用户修改，让下一次生成学习本项目偏好。
- 支持版本对比：AI 初稿、用户修改稿、最终稿。
- 支持一键保存到资产库、一键加入审阅、一键导出 Markdown/Docx/CSV。

4.7 社区和模板体验

社区不应早期做成泛内容 feed，应先服务生产效率：

- 模板库优先于社区广场。
- 模板按行业、渠道、目标、内容类型分类。
- 每个模板展示适用场景、输入要求、输出样例、预计成本、fork 次数。
- 组织内部模板和公共模板分开。
- 公共模板发布需要审核，避免低质内容污染。
- 社区作品可以作为灵感案例，但默认不进入用户品牌记忆，必须用户主动收藏。

4.8 可访问性与多端适配

最低要求：

- 全站键盘可操作，常用操作有快捷键。
- 所有图标按钮必须有 tooltip 或 aria-label。
- 表单错误要定位到字段，不只显示顶部 toast。
- 移动端优先支持查看、审核、评论、轻量生成；复杂画布以桌面端为主。
- 长文本不能溢出按钮、卡片、节点和侧栏。
- 深色/浅色主题需要在内容密集场景下验证对比度。

5. 核心技术栈与技术路线

5.1 总体架构路线

建议采用“模块化 Django 单体 + 强边界领域 app + 可替换基础设施”的路线，而不是过早微服务化。

原因：

- 当前团队和项目阶段更需要快速迭代、统一事务和低运维复杂度。
- 现有 Django app 边界已经有基础，继续强化即可。
- 真正需要独立扩展的是 AI 任务执行、模型网关、对象存储、检索索引和观察系统，不是所有业务都拆服务。

目标架构：

- 前端：React 应用，按业务域拆页面和状态。
- API 层：Django REST Framework，按 domain app 管理视图和权限。
- 任务层：Celery + Redis 作为当前主路线，复杂 DAG 规模扩大后评估 Temporal。
- 数据层：PostgreSQL 作为主库，Redis 做缓存/队列，pgvector 或 Qdrant 做向量检索。
- 媒体层：S3/OSS/R2 + CDN + 预签名 URL。
- AI 层：统一 Model Gateway，适配 OpenAI、Anthropic、Gemini、本地或代理模型。
- 运维层：Docker Compose 本地，生产使用容器平台、CI/CD、日志、监控、告警。

5.2 前端技术路线

保留：

- React、Vite、TypeScript、Tailwind CSS、lucide-react。

新增：

- React Router：建立稳定路由结构，支持项目深链和分享链接。
- TanStack Query：统一 API 请求、缓存、轮询、失效和错误重试。
- Zustand 或 Jotai：仅管理轻量 UI 全局状态，如主题、侧栏、选中项目。
- React Hook Form + Zod：表单状态和运行时校验，前后端 schema 可逐步对齐。
- XYFlow/React Flow：替代手写工作流画布。
- Radix UI 或 Headless UI：提供无障碍弹窗、菜单、Tabs、Popover、Tooltip。

- Playwright: 端到端测试和关键路径截图验证。
- Vitest + Testing Library: 组件和 hook 测试。

前端拆分建议:

- `src/app`: 路由、应用壳、provider。
- `src/features/projects`: 项目列表、项目详情、品牌记忆。
- `src/features/workflows`: 画布、节点、模板、运行记录。
- `src/features/generation`: 内容包生成、任务状态、结果编辑。
- `src/features/assets`: 资产库、预览、标签、导出。
- `src/features/community`: 模板库、灵感案例。
- `src/features/billing`: 套餐、用量、发票。
- `src/features/settings`: 组织、成员、模型配置、安全。
- `src/shared`: API client、UI 基础组件、types、utils。

5.3 后端技术路线

保留:

- Django 6、Django REST Framework、Celery、Redis、PostgreSQL。
- 继续维持领域 app 拆分，避免把新业务写回 `api.views`。

新增或强化:

- 权限: Django auth + DRF permission classes + organization-scoped queryset。
- Token: 优先使用 HttpOnly session 或 SimpleJWT，视部署场景选择；不要继续使用 demo token。
- RBAC: 基于 `Membership.role` 实现 admin/creator/ops/viewer 权限矩阵。
- 序列化: 继续集中在 serializer 层，减少 view 里手写 dict。
- 服务层: 复杂业务进入 service，不在 APIView 中堆流程。
- API Schema: 引入 OpenAPI 生成，前端类型可从 schema 生成。
- 幂等性: 生成任务、支付回调、上传回调需要 idempotency key。

- 速率限制：DRF throttle 或 API gateway 级限流。
- 审计日志：关键操作写 `AuditLog`，包括登录、成员变更、密钥变更、导出、删除、计费变更。

5.4 AI 与模型网关路线

模型调用必须从业务 view 中抽离为统一网关：

- Provider adapter：OpenAI、Anthropic、Gemini、mock、本地代理。
- Capability registry：标记模型支持 text、vision、image、audio、embedding、function calling。
- Model policy：按组织方案、用户角色、任务类型选择模型。
- Cost calculator：按 provider/model/token/image/audio 时长计算成本。
- Retry policy：超时、限流、可重试错误、不可重试错误分级处理。
- Safety policy：敏感词、品牌禁用词、平台规则、内容审核。
- Prompt registry：系统 prompt、节点 prompt、模板 prompt 版本化。
- BYOK isolation：组织自己的密钥只能用于该组织任务。
- Fallback policy：主模型失败时可切换备选模型，但要向用户说明。

5.5 向量检索与品牌记忆路线

建议分两步：

第一步使用 pgvector：

- 运维简单，直接在 PostgreSQL 中管理 embedding chunks。
- 适合早期组织规模和文档量。
- 支持项目级、组织级、模板级检索。

第二步评估 Qdrant：

- 当文档量、检索 QPS、多模态 embedding、过滤条件复杂度上升后迁移。
- 保留 repository/adaptor 层，避免业务绑定具体向量库。

品牌记忆数据来源：

- 项目 brand_context。
- 用户上传的品牌手册、历史文案、产品资料、FAQ。
- 用户收藏的社区/模板案例。
- 用户对生成结果的修改和最终稿。
- 禁用词、合规词、渠道规则。

检索返回要求：

- 不只返回结果，还要返回引用来源、相似度、更新时间、所属项目。
- 生成结果需要能解释“参考了哪些历史素材”。
- 默认不跨组织检索。

5.6 workflow 执行路线

当前阶段：

- 保持 Celery + Redis，新增 WorkflowRun 和 WorkflowRunNode 表，将一次运行固化。
- 每个节点运行生成独立 task 或 task step，记录输入快照、输出快照、错误、耗时、成本。
- 支持失败节点重试和从某节点继续运行。

中期：

- 使用 Celery canvas/chord/group 支持并行节点。
- 引入条件节点和人工审核节点。
- 增加 workflow 版本和发布态。

长期：

- 当 workflow 运行涉及多小时等待、人工节点、跨服务补偿时，评估 Temporal。
- 不建议当前立即引入 Temporal，避免过度工程化。

5.7 数据模型升级建议

优先新增或调整的模型：

- `Folder`：替代 `Project.folder_path` 字符串，支持层级和排序。
- `Tag`：组织级标签，区分 platform/status/content/custom。
- `OrganizationSetting`：组织配置、默认语言、默认模型、数据保留策略。
- `EncryptedCredential`：组织级 provider 密钥，加密保存和轮换。
- `ModelProvider` / `ModelProfile`：模型能力、价格、默认参数。
- `QuotaLedger`：额度发放、消耗、退款、调整。
- `WorkflowDefinition`： workflow 定义，与草稿和版本分离。
- `WorkflowVersion`：节点、边、schema、prompt 版本快照。
- `WorkflowRun`：一次运行记录。
- `WorkflowRunNode`：节点级运行记录。
- `NodeDefinition`：系统节点和自定义节点定义。
- `PromptTemplate`：prompt 版本化与变量定义。
- `AssetFile`：文件对象、mime、大小、hash、缩略图、存储 key。
- `KnowledgeDocument`：品牌知识文档。
- `EmbeddingChunk`：向量分块、embedding、来源引用。
- `ApprovalRequest`：审阅流。
- `Comment`：资产、内容包、workflow 节点评论。
- `ContentPackage`：一次生成的多平台内容包。
- `PublicationChannel`：渠道配置，早期先支持导出，不急于自动发布。
- `AuditLog`：安全和关键业务审计。
- `Experiment`：市场实验和功能开关。

5.8 基础设施与运维路线

开发环境：

- 保留 Docker Compose。

- 一键启动前后端、PostgreSQL、Redis、worker。
- 提供 seed 数据和 demo 组织。

测试环境：

- 独立数据库和对象存储 bucket。
- 自动部署每个主分支合并。
- 打开 debug 工具和详细日志，但不能使用生产密钥。

生产环境：

- 后端、worker、scheduler 分开部署。
- PostgreSQL 托管实例，开启自动备份和 point-in-time recovery。
- Redis 托管或高可用部署。
- S3/OSS/R2 对象存储 + CDN。
- Secret Manager/KMS 管理密钥。
- Sentry 捕获前后端异常。
- OpenTelemetry + Prometheus/Grafana 监控 API、任务、数据库、队列。
- 日志结构化，按 request_id、organization_id、task_id 关联。

5.9 安全路线

必须补齐：

- 关闭生产默认 debug。
- CORS 改为白名单。
- API Key 加密存储，绝不明文返回。
- 组织级数据访问检查。
- 成员角色权限矩阵。
- 登录保护、密码策略、会话过期。
- 速率限制和防刷。
- 上传文件大小、类型、病毒扫描和内容安全检查。

- 删除、导出、密钥变更写审计日志。
- 备份恢复演练。
- 隐私政策、用户协议、数据处理说明。

6. 分阶段实施计划

Phase 0: 稳定化与生产底线补齐, 1-2 周

目标: 让项目从 demo 工程进入可持续开发状态。

产品交付:

- 明确首批 ICP 和 3 个核心场景: 多平台图文、短视频脚本、品牌活动内容包。
- 梳理现有页面, 冻结非核心功能扩张。
- 制定设计规范: 颜色、间距、按钮、表单、空状态、错误状态、任务状态。

前端交付:

- 拆分 `App.tsx`, 按页面和 feature 组织代码。
- 引入 React Router 和 TanStack Query。
- 建立统一 API client、错误处理、toast、loading、empty state。
- 保留当前 UI 风格, 但移除普通用户界面的技术日志直出。

后端交付:

- 替换 demo token, 建立真实认证和请求用户解析。
- 为所有组织资源增加 organization-scoped 查询和权限检查。
- `AIConfiguration` 调整为组织级配置, 并规划密钥迁移。
- 生产环境默认关闭 debug 和 CORS all origins。
- 补齐 API 错误结构: `code`、`message`、`details`、`request_id`。

运维交付:

- CI 跑 `manage.py check`、后端测试、前端 lint、前端 build。
- 增加 `.env.example`，区分 dev/staging/prod。
- 增加基础 Sentry 或日志 `request_id`。

验收标准：

- 主分支每次合并自动跑基础检查。
- 用户无法跨组织读取项目、任务、资产。
- 登录态真实可校验。
- 关键 demo 路径仍可完成。

Phase 1: MVP 商业验证版，3-6 周

目标：打穿“品牌 brief -> 多平台内容包 -> 保存资产 -> 模板复用”的核心闭环。

产品交付：

- 新用户 onboarding。
- 项目详情页重构。
- 品牌记忆页。
- 内容包生成页。
- 资产库 MVP。
- 模板库 MVP。

前端交付：

- 项目列表支持列表/看板基础视图。
- 品牌上下文表单支持分组、校验和自动保存提示。
- 内容包生成支持多平台变体、结果编辑、保存版本。
- 资产库支持筛选、预览、标签、归档。
- workflow 画布迁移到 XYFlow/React Flow。
- 生成任务状态统一组件：排队、运行、成功、失败、重试。

后端交付：

- `ContentPackage`、`AssetFile`、`Tag`、`Folder` 模型。
- 对象存储接入，支持预签名上传和生成产物入库。
- Model Gateway MVP: mock + 至少一个真实文本模型 provider。
- 用量账本升级：每次生成写成本、模型、耗时、组织、项目、成员。
- workflow 运行记录：`WorkflowRun`、`WorkflowRunNode`。
- 模板 fork 和版本快照稳定化。

AI 交付：

- 品牌上下文注入 prompt。
- 多平台内容包 prompt 模板。
- 失败重试和 JSON 输出校验。
- 模型输出 schema 校验，失败时自动修复一次。

市场交付：

- 招募 20-50 个封闭测试组织。
- 每周访谈 5-8 个活跃用户。
- 建立反馈看板和问题优先级。

验收标准：

- 新用户 8 分钟内完成第一份内容包。
- 生成任务成功率大于 95%。
- 至少 30% 测试组织保存一个模板或复用一次历史资产。
- 核心路径有 Playwright E2E。

Phase 2：团队协作与付费闭环，7-10 周

目标：从个人生产工具升级为团队可付费协作工具。

产品交付：

- 成员邀请、角色管理。
- 内容审阅流。
- 评论和修改记录。
- 套餐页、用量页、额度告警。
- BYOK 配置向导。

前端交付：

- 成员管理页。
- 审阅队列和评论面板。
- 版本对比。
- 计费用量看板。
- 密钥配置体验：测试连接、权限提示、失败解释。

后端交付：

- RBAC 权限矩阵落地。
- `ApprovalRequest`、`Comment`、`QuotaLedger`、`AuditLog`。
- Stripe 或国内支付服务接入方案选型和 MVP。
- 额度扣减、超额阻断、管理员告警。
- BYOK 密钥加密存储、轮换、删除。

AI 交付：

- 品牌记忆检索 MVP：pgvector + embedding。
- 生成结果引用历史素材来源。
- 审核节点：品牌禁用词、渠道规则、敏感表达。

市场交付：

- 测试 Pro/Team 价格页。
- 设计试用转付费路径。
- 收集愿付价格和阻碍因素。

验收标准：

- 团队成员权限可控。
- 额度扣减准确，可追踪到任务。
- 密钥不明文展示，删除后不可使用。
- 至少 10 个组织完成团队协作路径测试。

Phase 3：稳定性、规模化与企业基础，11-16 周

目标：支持更大规模组织和更高可信度交付。

产品交付：

- 组织级模板库。
- 高级品牌知识库。
- 批量生成。
- 管理员控制台。
- 导出中心。

前端交付：

- 大量资产和任务的分页/虚拟列表。
- 批量操作体验。
- 高级搜索。
- 管理员仪表盘。
- 关键页面视觉回归。

后端交付：

- 队列分优先级：交互任务、批量任务、低优先任务。
- 缓存策略。
- 异步导出。
- 备份恢复和数据保留策略。

- OpenAPI 完整发布。
- 性能压测和慢查询治理。

运维交付：

- SLO：API p95、任务排队时间、生成成功率。
- Prometheus/Grafana 仪表盘。
- 队列积压告警、任务失败告警、成本异常告警。
- 数据库自动备份和恢复演练。

验收标准：

- 常规 API p95 小于 500ms。
- 任务状态推送或轮询稳定，用户不会丢失结果。
- 队列积压可观测、可告警。
- 可支持至少 100 个活跃组织的封闭/半公开测试。

7. 团队分工建议

产品负责人：

- 定义 ICP、核心场景、定价实验、版本优先级。
- 维护需求池和验收标准。
- 每周输出用户访谈结论。

设计负责人：

- 建立设计系统和交互规范。
- 重点负责 onboarding、项目工作台、生成结果编辑、 workflow 画布。
- 做可用性测试，记录任务完成率和卡点。

前端负责人：

- 完成架构拆分、路由、请求缓存、画布迁移。

- 建立组件测试和 E2E 测试。
- 控制 UI 一致性和复杂状态。

后端负责人：

- 负责权限、组织隔离、API schema、数据模型、服务层。
- 负责任务运行、用量账本、对象存储和模型网关。
- 控制数据库迁移质量。

AI 工程负责人：

- 负责 prompt 模板、模型适配、输出 schema、RAG、内容审核。
- 建立生成质量评测集。
- 跟踪成本、成功率和模型切换策略。

DevOps/运维负责人：

- 负责 CI/CD、环境隔离、密钥管理、监控、日志、告警、备份。
- 建立生产发布和回滚流程。

QA 负责人：

- 建立核心路径测试矩阵。
- 维护回归用例、兼容性测试、E2E 测试。
- 每次发版给出风险报告。

市场增长负责人：

- 招募封闭测试用户。
- 组织访谈、收集案例、验证定价。
- 打磨 landing page 和销售材料，但不干扰产品主屏体验。

8. 关键指标体系

产品指标：

- 首次价值时间小于 8 分钟。
- 新组织 7 日留存大于 25%。
- 活跃组织每周至少创建 2 个内容包。
- workflow 或模板复用率大于 30%。
- 资产复用率大于 20%。
- 团队邀请率大于 20%。

质量指标：

- 生成任务成功率大于 95%。
- 失败任务可解释率大于 95%。
- 核心 API p95 小于 500ms。
- 任务状态延迟小于 3 秒。
- 前端核心路径 E2E 覆盖率覆盖登录、创建项目、生成内容包、保存资产、运行 workflow。

商业指标：

- 试用到激活转化率。
- 激活到付费意向比例。
- 每组织平均模型成本。
- BYOK 使用比例。
- Pro/Team 转化率。
- 客户支持问题数量和类型。

安全指标：

- 跨组织访问测试通过率 100%。
- 密钥明文泄露检查 0。
- 高危依赖漏洞 0。
- 关键操作审计覆盖率 100%。

9. 风险与应对

产品风险：用户只把它当成一次性生成器，无法形成留存。

应对：核心体验围绕项目、品牌记忆、资产库、模板复用，而不是单次生成。

竞争风险：通用大模型和设计工具快速复制单点能力。

应对：建立垂直营销 workflows、团队协作、品牌资产和成本治理壁垒。

技术风险：工作流能力过早复杂化导致交付延期。

应对：先支持线性/简单 DAG、节点重试和版本快照，条件分支和人工节点放到 Phase 2/3。

成本风险：模型调用成本不可控。

应对：用量账本、预算上限、模型分层、BYOK、缓存、批量任务限速。

安全风险：用户密钥和品牌资料敏感。

应对：组织隔离、密钥加密、审计日志、最小权限、数据删除策略。

质量风险：AI 输出不稳定影响信任。

应对：结构化输出校验、自动修复、人工审核节点、品牌规则检查、评测集。

运维风险：任务队列积压或 worker 故障导致结果丢失。

应对：持久化任务状态、幂等运行、重试策略、队列监控、失败恢复。

10. 近期优先级清单

P0，必须立即做：

- 替换 demo 登录和 token。
- 组织级权限和数据隔离。
- AI 密钥组织级加密存储。
- 生产配置安全化。
- CI 基础检查。
- 前端拆分和统一 API 状态管理。

P1，直接影响市场验证：

- 内容包生成闭环。
- 品牌记忆表单。
- 对象存储和资产库。
- workflow 画布迁移到 React Flow。
- 任务状态和失败重试体验。
- 模板库 MVP。

P2, 影响付费:

- 用量额度和成本看板。
- BYOK 配置向导。
- 成员邀请与 RBAC。
- 审阅流和评论。
- pgvector 品牌记忆检索。

P3, 影响规模化:

- 高级监报告警。
- 批量生成。
- 导出中心。
- 企业安全能力。
- OpenAPI 和前端类型生成。

11. 最终验收定义

当以下条件满足时，Marketing Hub 可以从“初版项目”进入“小规模商业验证”：

- 用户能独立完成：创建项目、填写品牌记忆、生成多平台内容包、保存资产、复用模板。
- 团队能完成：邀请成员、分配角色、审阅内容、评论修改、导出最终稿。
- 管理员能完成：配置模型、查看用量、控制额度、管理密钥、查看审计日志。
- 系统能保证：组织数据隔离、任务可追踪、失败可重试、成本可计算、资产可持久化。

- 市场能验证：至少 20 个目标组织完成封闭测试，出现明确付费意愿和高频使用场景。

12. 结论

Marketing Hub 的方向值得继续推进，但升级重点必须从“多功能展示”转向“可信生产闭环”。当前项目最有价值的资产是已经建立了组织、项目、活动、工作流、任务、资产、社区、计费的基础骨架；最危险的地方是安全、权限、真实 AI 产物、检索、计费和运维还没有达到生产标准。

团队后续开发应坚持一个原则：每新增一个 AI 能力，都必须回答它如何进入项目、如何保存为资产、如何参与品牌记忆、如何计费、如何被审阅、如何复用。只有这样，产品才能从一个好看的 AI Demo 变成经得起市场考验的营销生产平台。